

Raport z Projektu: Przewidywanie awarii maszyn (Predictive Maintenance)

Autorzy: Oliwier Trąbczyński, Bartosz Wankiewicz

1. Cel Biznesowy i Problem Badawczy

Głównym celem projektu było opracowanie modelu zdolnego do predykcji usterek maszyn przemysłowych (frezarek CNC) na podstawie danych telemetrycznych z czujników.

Rozwiązuje to fundamentalny problem utrzymania ruchu: przejście z kosztownej konserwacji reaktywnej (naprawa po zniszczeniu maszyny) na **Predictive Maintenance** (zapobieganie awariom z wyprzedzeniem). Analizie poddano syntetyczny zbiór AI4I 2020, symulujący realne parametry, takie jak temperatura procesu czy prędkość obrotowa.

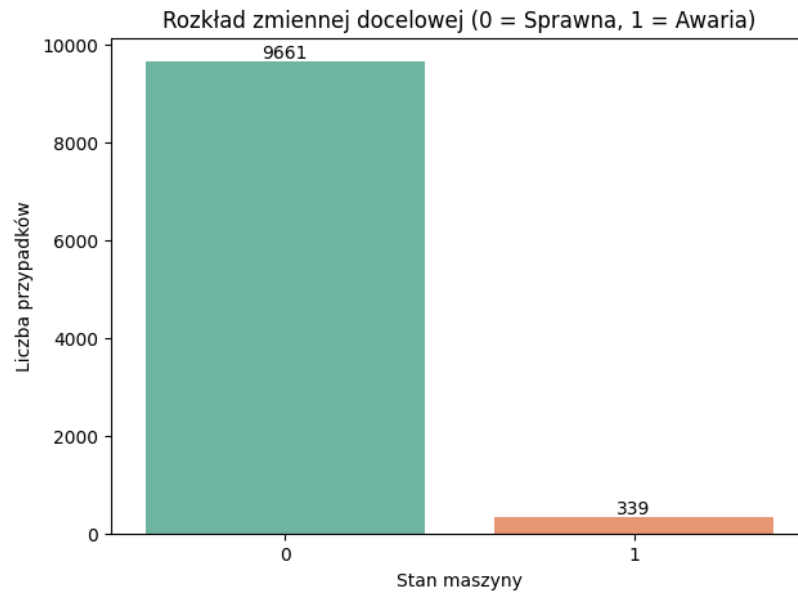
Hipoteza badawcza: Zastosowanie zaawansowanych modeli uczenia maszynowego opartych na zespołach drzew decyzyjnych (takich jak XGBoost) z odpowiednim wagowaniem klas, pozwoli na skuteczną predykcję rzadkich awarii maszyn z czułością (Recall) przewyższającą metody klasyczne, optymalizując tym samym koszty utrzymania ruchu.

2. Wyzwanie Techniczne: Paradoks Dokładności

Kluczową barierą w projekcie był drastyczny brak balansu klas.

- Maszyny sprawne stanowią 96.6% zbioru.
- Awarie stanowią zaledwie 3.4% zbioru.

W takich warunkach klasyczna metryka *Accuracy* (Dokładność) jest bezużyteczna. Algorytm, który całkowicie zignoruje odczyty i zawsze orzeknie "maszyna sprawna", osiągnąłby 96.6% dokładności, przegapiając przy tym wszystkie rzeczywiste usterki. Z tego powodu głównym celem optymalizacyjnym stała się **Czułość (Recall)** – zdolność modelu do wyłapywania jak największej liczby rzadkich anomalii.



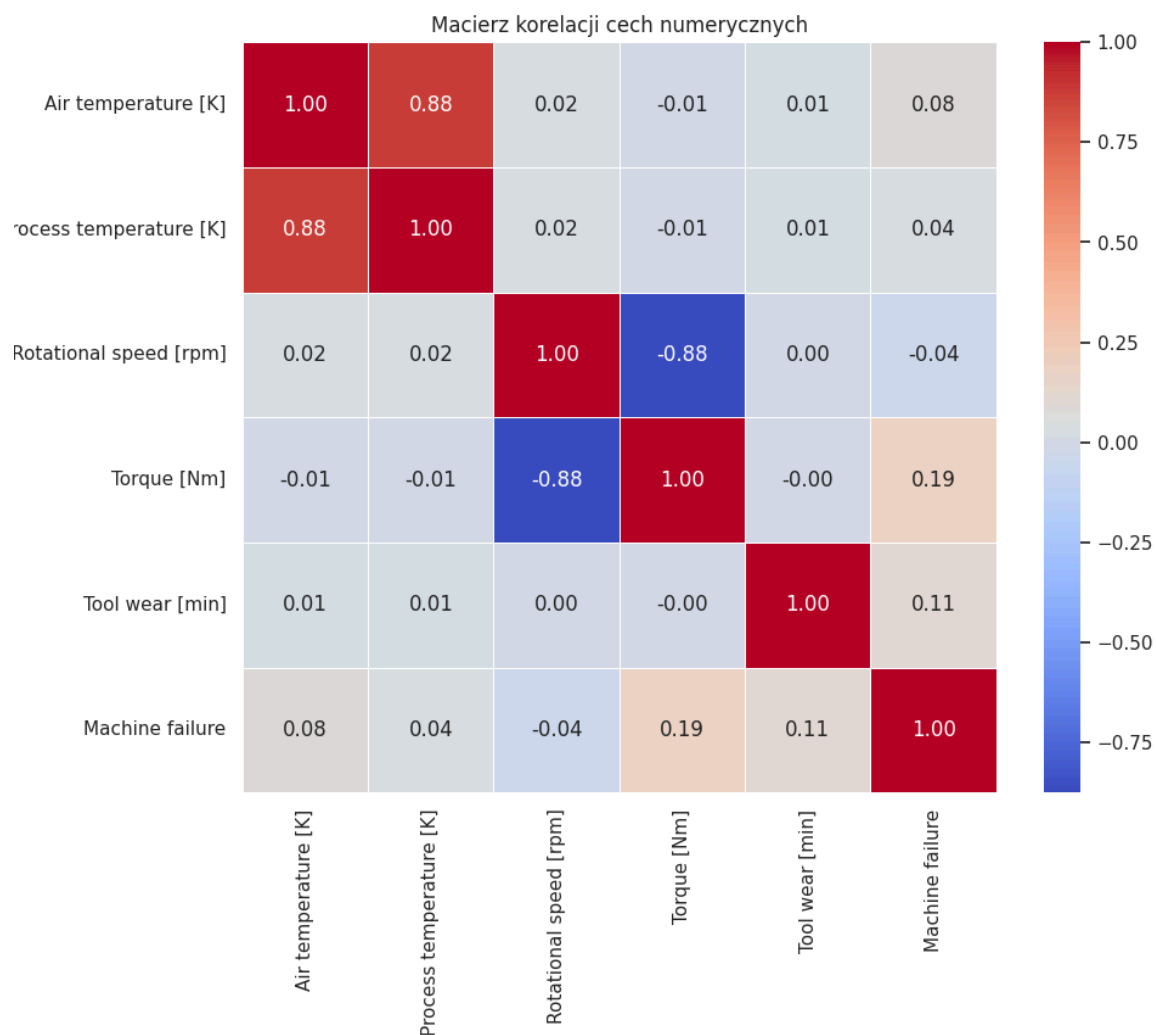
Analiza braków w danych (z wykorzystaniem funkcji isnull) wykazała, że zbiór jest w 100% kompletny. Z macierzy korelacji wynika silna zależność między zmienną docelową a momentem obrotowym i zużyciem narzędzia.

```
print("ANALIZA BRAKÓW W DANYCH")
missing_values = df.isnull().sum()
print(missing_values)
```

```
INFORMACJE O TYPAH ZMIENNYCH
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   UDI                                    10000 non-null  int64
1   Product ID                            10000 non-null  object
2   Type                                  10000 non-null  object
3   Air temperature [K]                   10000 non-null  float64
4   Process temperature [K]               10000 non-null  float64
5   Rotational speed [rpm]                10000 non-null  int64
6   Torque [Nm]                           10000 non-null  float64
7   Tool wear [min]                       10000 non-null  int64
8   Machine failure                       10000 non-null  int64
9   TWF                                    10000 non-null  int64
10  HDF                                    10000 non-null  int64
11  PWF                                    10000 non-null  int64
12  OSF                                    10000 non-null  int64
13  RNF                                    10000 non-null  int64
dtypes: float64(3), int64(9), object(2)
```

```
memory usage: 1.1+ MB
ANALIZA BRAKÓW W DANYCH
```

```
UDI          0
Product ID   0
Type         0
Air temperature [K]  0
Process temperature [K]  0
Rotational speed [rpm]  0
Torque [Nm]  0
Tool wear [min]  0
Machine failure  0
TWF          0
HDF          0
PWF          0
OSF          0
RNF          0
dtype: int64
```



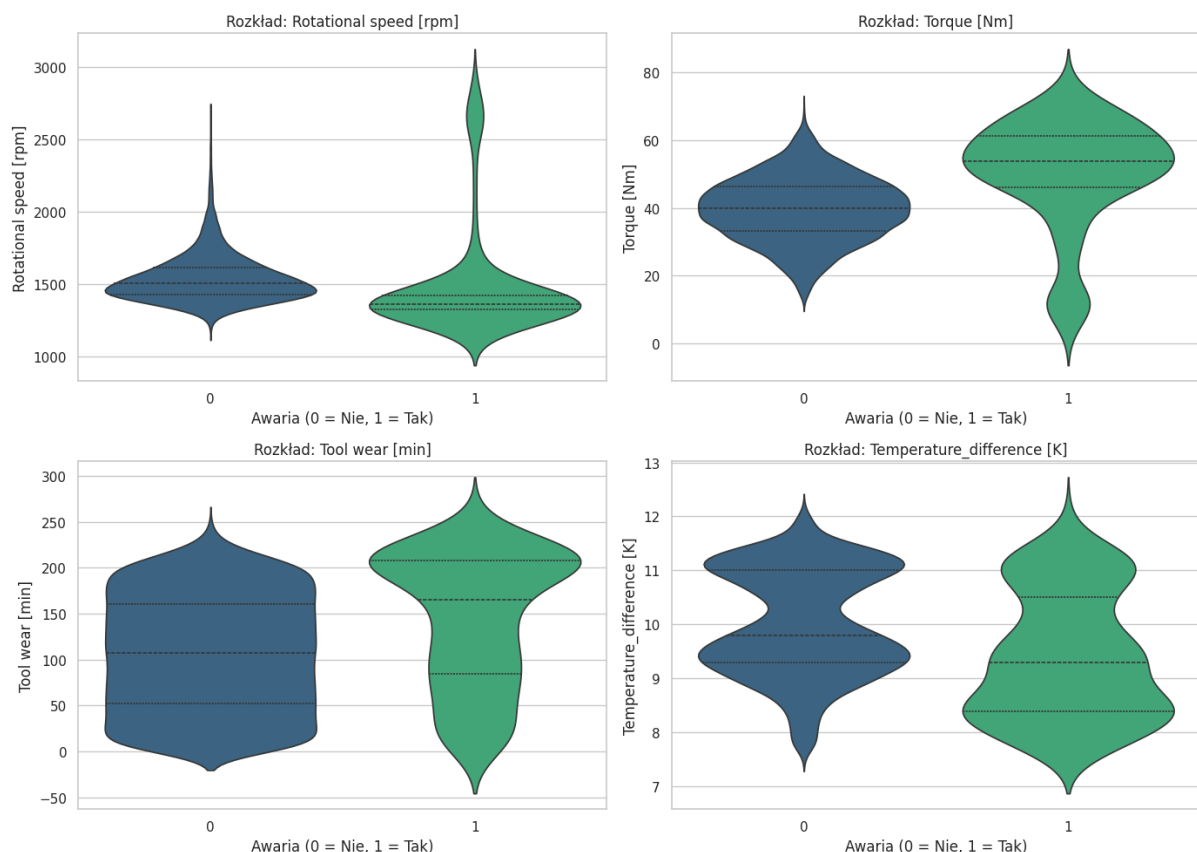
3. Aktualny stan wiedzy:

1. **Klasyczne algorytmy nadzorowane (Supervised Learning):** Wykorzystanie modeli takich jak Random Forest, XGBoost czy Support Vector Machines (SVM) do binarnej klasyfikacji stanu maszyny na podstawie zagregowanych cech (np. temperatura, vibracje). Modele te świetnie radzą sobie z danymi tabelarycznymi i potrafią modelować nieliniowe zależności, co czyni je najczęstszym wyborem w przemyśle.
2. **Głębokie uczenie (Deep Learning) dla szeregów czasowych:** Wykorzystanie sieci rekurencyjnych (RNN), w szczególności architektury LSTM (Long Short-Term Memory) lub GRU, do analizy sekwencyjnych danych telemetrycznych. Podejście to pozwala na bezpośrednie prognozowanie RUL (Remaining Useful Life - pozostałego czasu bezawaryjnej pracy).
3. **Detekcja anomalii (Unsupervised Learning):** W sytuacjach, gdzie dane o awariach są skrajnie rzadkie (ogromna dysproporcja klas), stosuje się modele nienadzorowane, takie

jak Isolation Forest, One-Class SVM lub Autoenkodery. Uczą się one "normalnego" profilu pracy maszyny i oflagowują każde odchylenie jako potencjalną anomalię.

4. Analiza Eksploracyjna: Fizyczny obraz awarii

Przed przystąpieniem do budowy modeli predykcyjnych, kluczowe było zrozumienie natury samych awarii. W tym celu przeprowadzono analizę rozkładów najważniejszych zmiennych ciągłych w podziale na stan maszyny (sprawna vs awaria).



Wnioski z analizy wizualnej: Rozkłady gęstości jednoznacznie potwierdzają, że awarie nie mają charakteru losowego – są silnie skorelowane ze skrajnymi odczytami z czujników telemetrycznych:

- **Prędkość obrotowa (Rotational speed):** Rozkład dla maszyn uszkodzonych jest spłaszczony i rozciągnięty. Awarie korelują ze skrajnie niskimi lub bardzo wysokimi prędkościami, podczas gdy maszyny sprawne pracują w stabilnym, wąskim przedziale.
- **Moment obrotowy (Torque):** Awarie występują przy znacznie wyższej wariancji (rozzrucie) siły skręcającej, co sugeruje trudności w obróbce twardego materiału.
- **Zużycie narzędzia (Tool wear):** To najsilniejszy wizualny wskaźnik. Mediana czasu pracy dla maszyn, które uległy awarii, jest drastycznie przesunięta w górę. Zbyt długa praca jednym narzędziem (np. tępym frezem) krytycznie zwiększa ryzyko usterki.

Dzięki tej analizie zyskaliśmy pewność, że w danych ukryty jest silny, nieliniowy wzorec, z którym doskonale powinny poradzić sobie modele oparte na drzewach decyzyjnych.

5. Wybór i mechanika Modelu: Las Losowy vs XGBOOST

Odrzuceni kandydaci:

1. Maszyny Wektorów Nośnych (Support Vector Machines - SVM)

- **Sposób działania:** Algorytm szuka optymalnej hiperpłaszczyzny w wielowymiarowej przestrzeni, aby jak najlepiej oddzielić od siebie klasy (wykorzystując tzw. "kernel trick" dla danych nieliniowych).
- **Zalety:** Bardzo skuteczny przy złożonych, nieliniowych granicach decyzyjnych.
- **Wady:** Wysoce wrażliwy na brak skalowania danych (wymagałby ścisłej standaryzacji np. obrotów silnika), a trenowanie na większych zbiorach trwa stosunkowo długo.
- **Dopasowanie do problemu:** Teoretycznie potrafiłby odseparować awarie od poprawnej pracy, jednak duży narzut obliczeniowy oraz trudność w odpowiednim ważeniu klas czynią go mniej praktycznym wyborem.

2. Regresja Logistyczna (Logistic Regression)

- **Sposób działania:** Klasyczny model statystyczny szacujący prawdopodobieństwo przynależności do danej klasy za pomocą funkcji logistycznej (sigmoidy).
- **Zalety:** Szybki w trenowaniu, lekki obliczeniowo i w pełni interpretowalny (wagi pokazują wpływ cech).
- **Wady:** Zakłada liniową zależność między zmiennymi objaśniającymi a zmienną celu.
- **Dopasowanie do problemu:** Rozpatrywany wyłącznie jako prosty model bazowy (baseline). Złożona fizyka pracy frezarki (relacje między momentem obrotowym, zużyciem narzędzia i temperaturą) sprawia, że modele czysto liniowe nie są w stanie skutecznie zmapować anomalii zwiastujących awarię.

W fazie implementacji zderzono ze sobą dwa potężne algorytmy oparte na zespołach drzew decyzyjnych: Random Forest (wykorzystujący mechanizm baggingu) oraz XGBoost (wykorzystujący boosting). W obu modelach zastosowano techniki wagowania klas (class_weight oraz scale_pos_weight), aby zmusić algorytmy do kładzenia nacisku na rzadko występujące awarie.

```

=====
WYNIKI DLA MODELU: XGBOOST
=====
              precision    recall  f1-score   support

      0       0.99         0.99         0.99        1932
      1       0.82         0.82         0.82         68

   accuracy                0.99        2000
  macro avg              0.91         0.91         0.91        2000
 weighted avg              0.99         0.99         0.99        2000

=====
WYNIKI DLA MODELU: RANDOM FOREST (Las Losowy)
=====
              precision    recall  f1-score   support

      0       0.99         1.00         0.99        1932
      1       0.94         0.69         0.80         68

   accuracy                0.99        2000
  macro avg              0.96         0.84         0.90        2000
 weighted avg              0.99         0.99         0.99        2000

```

Wnioski z porównania i mechanika algorytmu: Mimo zastosowania zbalansowanych wag, Random Forest okazał się niewystarczający w warunkach silnego niezbalansowania danych – osiągnął Czułość (Recall) dla klasy awarii na poziomie jedynie 0.69 (przeoczył aż 31% usterek). Z kolei XGBoost (Model Bazowy) poradził sobie znacznie lepiej, osiągając Czułość rzędu 0.82.

Dlaczego XGBoost wygrał to starcie? Zamiast budować jeden skomplikowany model, tworzy on zespół (sztafetę) wielu prostych drzew decyzyjnych. Każde kolejne drzewo w sekwencji uczy się na błędach swojego poprzednika, skupiając się na tych maszynach, które wcześniej zostały źle sklasyfikowane. Sekwencyjny sposób uczenia algorytmu XGBoost okazał się idealny do wyłapywania trudnych, nieliniowych anomalii z czujników. Dzięki wbudowanemu parametrowi obsługi wag klas, model został skutecznie zmuszony do priorytetyzowania mniejszościowych przypadków awarii. Z tego względu to on został wytypowany jako model docelowy.

6. Optymalizacja Hiperparametrów (GridSearchCV)

Mając wybranego faworyta (XGBoost), podjęto próbę wyśrubowania jego zdolności do wyłapywania awarii, wykorzystując metodę przeszukiwania siatki parametrów (GridSearchCV).

Kluczowe założenia eksperymentu:

1. Zastosowano walidację krzyżową ze stratyfikacją (**StratifiedKFold**), co zagwarantowało, że model podczas uczenia w każdym z podzbiorów testowych widział dokładnie taki sam odsetek awarii (3.4%).

2. Siatka była optymalizowana bezpośrednio pod kątem maksymalizacji metryki **Recall**, a nie domyślnej dokładności.

Algorytm przetestował dziesiątki kombinacji i wyłonił architekturę opartą na płytkich drzewach (**max_depth=3**) oraz niskim tempie uczenia (**learning_rate=0.01**). Ta modyfikacja pozwoliła na drastyczny skok skuteczności: model zoptymalizowany podniósł Czułość z 82% do imponujących **93%**.

```
Najlepsze znalezione parametry: {'learning_rate': 0.01, 'max_depth': 3, 'n_estimators': 100}
```

```
=====
WYNIKI DLA ZOPTYMALIZOWANEGO MODELU XGBOOST
=====
```

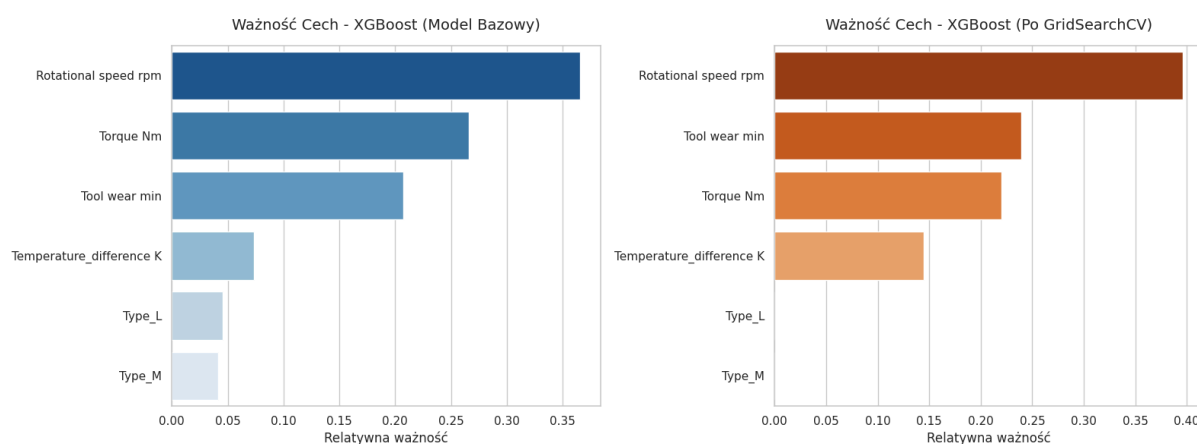
	precision	recall	f1-score	support
0	1.00	0.91	0.95	1932
1	0.26	0.93	0.40	68
accuracy			0.91	2000
macro avg	0.63	0.92	0.68	2000
weighted avg	0.97	0.91	0.93	2000

Wybór wariantów do ostatecznej ewaluacji: Od tego momentu, do finalnej analizy interpretacyjnej i biznesowej wytypowano dwa konkretne "wcielenia" zwycięskiego algorytmu, aby sprawdzić, jak optymalizacja wpłynęła na jego decyzje:

- **XGBoost (Model Bazowy):** Korzystający z domyślnych ustawień, stanowiący stabilny punkt odniesienia.
- **XGBoost (Po GridSearchCV):** Ekstremalnie "czuły", wyśrubowany algorytm ratujący najwięcej maszyn.

7. Interpretowalność: Co faktycznie powoduje awarie?

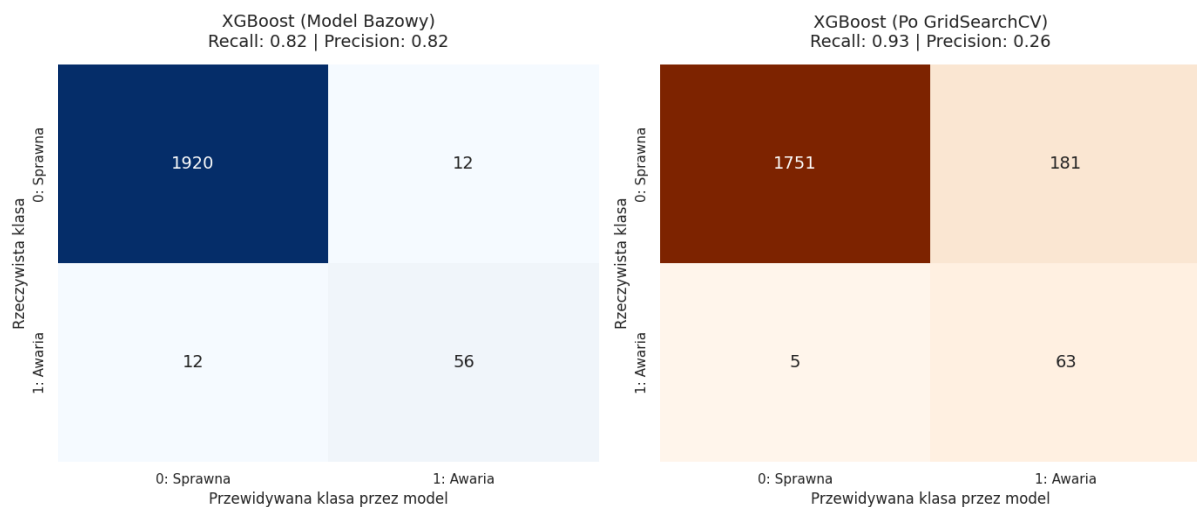
Wiedza o tym, *dlaczego* model podejmuje decyzje, jest kluczowa dla inżynierów. Przeprowadzono analizę ważności cech (Feature Importance) przed i po optymalizacji hiperparametrów za pomocą metody GridSearchCV.



Wnioski z wykresu: Model zoptymalizowany (wykorzystujący płytkie drzewa o głębokości równej 3) musiał dokonać ostrej selekcji informacji. Całkowicie zignorował on categoryczne etykiety jakości producenta, redukując ich wagę do zera. Algorytm udowodnił, że twarda fizyka z czujników (prędkość obrotowa, zużycie narzędzia tnącego, moment obrotowy i różnica temperatur) jest w zupełności wystarczająca do przewidzenia zbliżającej się awarii.

8. Kompromis Biznesowy: Precyzja vs Czułość

Ostatnim etapem projektu była ewaluacja biznesowa. W uczeniu maszynowym zwiększenie Czułości (wykrywanie większej liczby awarii) zawsze pociąga za sobą spadek Precyzji (generowanie fałszywych alarmów).



Macierze pomyłek obrazują wynik działania dwóch strategii na zbiorze testowym:

- **Strategia 1 (Model Bazowy):** Wyłapuje 82% awarii, generując przy tym minimalną liczbę fałszywych alarmów. To stabilny "złoty środek".
- **Strategia 2 (Model Zoptymalizowany):** Działa jak czuły alarm dymny. Wyłapuje aż 93% awarii, ale kosztem drastycznego wzrostu fałszywych alarmów (precyzja spada do 26%).

9. Symulacja Finansowa i Wniosek Końcowy

Czy warto było tak drastycznie obniżać precyzję modelu zoptymalizowanego? Odpowiedź daje symulacja kosztów operacyjnych.

Zakładając koszt krytycznej usterki (np. spalanie wrzeczona) na poziomie 15 000 PLN oraz koszt weryfikacji fałszywego alarmu na 200 PLN, otrzymujemy następujące straty dla rozpatrywanej partii maszyn:

- **Strata przy modelu bazowym:** 182 400 PLN (przez większą liczbę przeoczonych awarii).
- **Strata przy modelu zoptymalizowanym:** 110 800 PLN (ratujemy więcej maszyn kosztem tanich wizyt technika).

Rekomendacja końcowa: Hipoteza badawcza została potwierdzona. Modele nie muszą być perfekcyjne we wszystkich metrykach. Celowe podbicie Czułości na rzadkich danych telemetrycznych pozwala na zaoszczędzenie ponad 70 000 PLN, udowadniając, że dobrze skalibrowany model to bezpośrednie narzędzie optymalizacji finansowej zakładu.